

AM50 COMPUTER LAB 10

1. INTRODUCTION

The goal of today's computer lab is as follows:

- (1) Learn about how to count the frequency distributions of words in a passage of text.
- (2) Compare the frequency distributions in a few (famous and not so famous) texts, and compare your measurements to Zipf's law.
- (3) Use this information to measure *Shannon's entropy* for these texts.

2. SHANNON'S ENTROPY, AND THE MAIN POINT OF THE LAB

Our paper this week is Shannon's Mathematical Theory of Information, in which Shannon discovers how to measure the information content of, e.g. an English text. At the heart of Shannon's idea is that we should think of a text as emanating from an information source, which *probabilistically* emits letters or words at some pre-determined frequencies.

Namely, when writing the term paper for your Literature class, Shannon envisions you as having some set of words at your disposal, and your brain chooses words from this distribution and puts them on the page. To the extent that you are choosing your words in a not-random fashion, the particular order of the words in your brain is the information content of the document.

Shannon measures the information through the *entropy* of the text. Suppose that the your brain contains M different words, and that the i^{th} word has a frequency f_i . Then the entropy is given by

$$(1) \quad H = - \sum_{i=1}^M f_i \log_2(f_i).$$

Note that the logarithm here is base 2¹

As Shannon argues, if you then consider writing a document of N words, the number of possible combinations of words you can choose is

$$\# \text{ of Words} = 2^{HN}.$$

Thus the particular combination of words that you have chosen for your essay is one out of this many possibilities. Note that the higher the entropy, H , the more possible documents your brain can put together—and the more unique each particular one is! Shannon defines H with respect to the base 2 logarithm because this way you can imagine coding all of the words in a binary counter—ie with a series of 0's and 1's, like a computer. Then the entropy H measures the number of binary digits required to code a word of the language.

So the question is: What is H ? What is your personal H ? What is the H for the English language? Are they all the same? Or do better (or worse) writers have a higher value for H ?

The goal of today's lab is to make some simple measurements to begin to answer these questions.

3. COUNTING WORD FREQUENCY DISTRIBUTIONS

Our goal is to give you the needed tools to compute the entropy of a text. Once you can do this you can measure the entropy of a few representative texts that we

¹This, and today's entire lab, is a simplification of the real calculation. In reality, the statistical structure of a language does not just depend on the word frequencies but on the relationship of different words to each other. So for example, a sentence beginning with the word "So" is more likely to have a second word "for" than "example". We are neglecting this effect in today's lab. If you are interested you could turn a more precise analysis into a final project!"

are going to give you. In your homework you will measure the entropy of your own writing.

To count the word frequencies, you should use the MATLAB program *wordcount*, that is posted on the course web site. This program was written by a contributor to MATLAB's public code repository, and takes a filename as input and then outputs the word frequency distribution. We have modified the program a little so that it allows you to plot the outputted data.

The program contains several concepts that are beyond the level of this class and it is not necessary that you understand it in detail. However we will illustrate how a few of the commands work so that you get the idea.

- (1) First it opens the file. The file **must** be a 'txt' file—it cannot read in word files for examples. You can easily make a txt file by copying the bit of text, and then saving it as a txt file within Microsoft Word. On the course web site we have given you a txt file of the Gettysburg Address, called "gettysburg.txt". Read this into MATLAB by typing

```
fid=fopen('gettysburg.txt');
```

This command opens the file and MATLAB assigns the file a file identifier *fid*.

- (2) Now the program converts the file into what is called a *cell array*. This is a data structure that allows MATLAB to easily count and manipulate the text in the file. To see what this is, type:

```
words=textscan(fid,'%s');
```

words is now a cell array, containing the words of the text. Type:

```
words{1}
```

MATLAB will spit out ALL of the individual words of the text file. Type

```
size(words{1})
```

and you will discover that this array has 278 elements—this is the total number of words in the Gettysburg address. The great advantage of cell arrays is that MATLAB has many built in functions operating on them, and this makes the rest of the program for counting word frequency distributions relatively easily.

- (3) For example, to count the number of unique words in the Gettysburg address you need to type

```
unique_words=unique(words{1})
```

This lists the number of unique words in the Gettysburg address, using the wonderful MATLAB command *unique*. (Type "help unique" to see what this does).

- (4) The actual program has a few more complicated steps—ie eliminating all symbol strings that are not words, and then counting them, but hopefully you can now see how the computer can possibly count word frequency distributions.

If you type

```
[results,freq]=wordcount('gettysburg.txt',100);
```

The number 100 tells the program you want the top 100 words in the word frequency count.

Exercise: 1

Use word count to compute the frequency distribution of words in the Gettysburg address. Plot the distribution on a log-log plot by typing `loglog(freq, '.')`

Now type

```
hold all
```

which does not let you overwrite the plot you just made.

We now want you to take two other documents and again measure the word frequency distribution, to see how it changes from document to document.

Exercise: 2

We have put a txt file of two recent news articles on the course web site, both of which discuss the non-government shut down that happened last week. The first article is from the Huffington post, and the second from Fox news. They have completely different slants as you would expect. Measure the word frequency distribution of this and plot it on top of the plot you made for the Gettysburg address.

Exercise: 3

Now make a text file of some article or document of interest to you, that you can find on the web. Make your document into a txt file (you can do this by cutting and pasting into Word and then saving it as a txt file). Measure the word frequency distribution of the document and plot it on your plot.

You should see that the word frequency distributions of your three documents have a surprising similarity. Compare the top few ranked words in the documents—are they the same or different? Look further down the list, when does the difference arise?

There is a famous law that governs the frequency distribution of words—called Zipf’s law. See

http://en.wikipedia.org/wiki/Zipf%27s_law

This says that the frequency distribution of words in a document obeys

$$f_k = \frac{C}{k},$$

where f_k is the frequency of occurrence of the k^{th} word and C is a constant.

Exercise: 4

Does your data agree with Zipf’s law? Why or why not?

4. CALCULATING THE ENTROPY

Finally, Shannon defined the entropy according to Equation (1).

Exercise: 5

Write a MATLAB function to compute the entropy of a set of word frequencies. The function should take in the array “freq” that is output by the ‘wordcounts’ program, and compute the entropy given in Equation (1). Make sure when you compute your entropy you use the base 2 logarithm, given by the MATLAB command *log2*.

Exercise: 5

Use your function to measure the entropy of each of the texts. Is there more information content in either the Fox news article or the Huffington post article?